

## ECA0905-DIGITAL SIGNAL PROCESSING

DATE:02-08-2026

### Aim

To simulate a multirate system using decimation and interpolation and analyze the sampling rate conversion.

### Given Data

- Input Sampling Rate = **18 kHz**
- Decimation Factor (M) = **3**
- Interpolation Factor (L) = **2**

### Tasks

1. Design an anti-alias filter.
2. Perform decimation.
3. Implement interpolation.
4. Analyze the output spectrum.
5. Determine the final sampling rate.
6. Compare input and output signals.

### Algorithm :

1. Start the MATLAB program.
2. Initialize the input parameters:
3. Input Sampling Rate  $F_s = 18\text{kHz}$
4. Decimation Factor  $M = 3$
5. Interpolation Factor  $L = 2$
6. Generate the input signal using a combination of sinusoidal signals.
7. Design an FIR low-pass anti-aliasing filter using the fir1 function.
8. Filter the input signal using the anti-aliasing filter to remove high-frequency components.
9. Perform Decimation by downsampling the filtered signal by a factor of 3 using the downsample() function.
10. Calculate the decimated sampling rate:

$$11. F_{dec} = \frac{F_s}{M}$$

12. Perform Interpolation by upsampling the decimated signal by a factor of 2 using the `upsample()` function.

13. Design a reconstruction low-pass FIR filter to eliminate imaging components introduced during interpolation.

14. Filter the interpolated signal using the reconstruction filter.

15. Calculate the final output sampling rate:

$$16. F_{out} = \frac{F_s \times L}{M}$$

17. Plot the time-domain signals:

18. Original input signal

19. Decimated signal

20. Interpolated signal

21. Final output signal

22. Compute the FFT of both the input and output signals.

23. Plot the input and output spectra for frequency-domain analysis.

24. Compare the input and output signals to observe the effect of sampling rate conversion.

25. Display the input sampling rate, decimation factor, interpolation factor, and final output sampling rate.

26. Stop the program.

### **Program :**

```
clc;

clear;

close all;

%% Given Data

Fs_in = 18000; % Input Sampling Frequency (Hz)

M = 3; % Decimation Factor

L = 2; % Interpolation Factor

%% Generate Input Signal

t = 0:1/Fs_in:0.02; % 20 ms duration
```

```

x = sin(2*pi*1000*t) + 0.5*sin(2*pi*3000*t);

%% Step 1: Anti-Alias Low Pass Filter

N = 64;           % Filter Order

Fc = 1/(2*M);     % Normalized Cutoff Frequency

h = fir1(N,Fc);

x_filtered = filter(h,1,x);

%% Step 2: Decimation

x_dec = downsample(x_filtered,M);

Fs_dec = Fs_in/M;

%% Step 3: Interpolation

x_int = upsample(x_dec,L);

%% Reconstruction Filter

Fc2 = 1/(2*L);

h2 = fir1(N,Fc2);

y = filter(h2,1,x_int);

%% Final Sampling Rate

Fs_out = Fs_dec*L;

fprintf('Input Sampling Rate = %.0f Hz\n',Fs_in);

fprintf('Output Sampling Rate = %.0f Hz\n',Fs_out);

%% Time Axis

t_out = (0:length(y)-1)/Fs_out;

%% Time Domain Plots

figure;

subplot(4,1,1)

plot(t,x)

title('Original Signal (18 kHz)')

xlabel('Time (s)')

ylabel('Amplitude')

grid on

```

```

subplot(4,1,2)
plot((0:length(x_dec)-1)/Fs_dec,x_dec)
title('After Decimation (6 kHz)')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

subplot(4,1,3)
plot((0:length(x_int)-1)/(Fs_dec*L),x_int)
title('Interpolated Signal')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

subplot(4,1,4)
plot(t_out,y)
title('Final Output Signal (12 kHz)')
xlabel('Time (s)')
ylabel('Amplitude')
grid on

%% Spectrum Analysis
NFFT = 2048;
X = fft(x,NFFT);
Y = fft(y,NFFT);
f1 = (0:NFFT-1)*(Fs_in/NFFT);
f2 = (0:NFFT-1)*(Fs_out/NFFT);

figure
subplot(2,1,1)
plot(f1(1:NFFT/2),abs(X(1:NFFT/2)))
title('Input Spectrum')
xlabel('Frequency (Hz)')
ylabel('Magnitude')

```

```

grid on
subplot(2,1,2)
plot(f2(1:NFFT/2),abs(Y(1:NFFT/2)))
title('Output Spectrum')
xlabel('Frequency (Hz)')
ylabel('Magnitude')

grid on

%% Signal Comparison

figure
plot(t,x,'b')
hold on
plot(t_out,y,'r')
legend('Input','Output')
title('Input vs Output Signal')
xlabel('Time (s)')
ylabel('Amplitude')

grid on

%% Verify Final Sampling Rate

disp('-----')
disp(['Input Sampling Rate = ',num2str(Fs_in),' Hz'])
disp(['Decimation Factor   = ',num2str(M)])
disp(['Interpolation Factor = ',num2str(L)])
disp(['Final Sampling Rate = ',num2str(Fs_out),' Hz'])
disp('-----')

```

Output:

